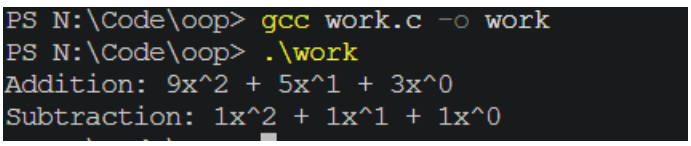
1.Use Linked List for polynomial representation and perform addition and subtraction operations on the polynomials

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int coeff,exp;
struct node *next;
};
struct node *create(int c,int e){
struct node *n=malloc(sizeof(struct node));
n->coeff=c;
n->exp=e;
n->next=NULL;
return n;
}
struct node *insert(struct node *head,int c,int e){
struct node *n=create(c,e),*t=head;
if(head==NULL)return n;
while(t->next!=NULL)t=t->next;
t->next=n;
return head;
}
struct node *add(struct node *a,struct node *b,int sign){
struct node *r=NULL;
while(a!=NULL&&b!=NULL){
if(a->exp==b->exp){
```

```
r=insert(r,a->coeff+sign*b->coeff,a->exp);
a=a->next;
b=b->next;
}else if(a->exp>b->exp){
r=insert(r,a->coeff,a->exp);
a=a->next;
}else{
r=insert(r,sign*b->coeff,b->exp);
b=b->next;
}
}
while(a!=NULL){
r=insert(r,a->coeff,a->exp);
a=a->next;
}
while(b!=NULL){
r=insert(r,sign*b->coeff,b->exp);
b=b->next;
}
return r;
}
void display(struct node *p){
while(p!=NULL){
printf("%dx^%d",p->coeff,p->exp);
if(p->next!=NULL)printf(" + ");
p=p->next;
```

```
}
printf("\n");
}
int main(){
struct node *p=NULL,*q=NULL,*r;
p=insert(p,5,2);
p=insert(p,3,1);
p=insert(p,2,0);
q=insert(q,4,2);
q=insert(q,2,1);
q=insert(q,1,0);
r=add(p,q,1);
printf("Addition: ");
display(r);
r=add(p,q,-1);
printf("Subtraction: ");
display(r);
return 0;
}
```

OUTPUT:



2.Find the previous node of Circular Linked List.

```
struct node{
int data;
```

```
struct node *next;
};
struct node *previous(struct node *head,int key){
struct node *temp;
if(head==NULL)return NULL;
temp=head;
do{
if(temp->next->data==key)return temp;
temp=temp->next;
}while(temp!=head);
return NULL;
}
```

3.Insert at a given position of a circular Linked List.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
struct node *next;
};
struct node *insert(struct node *head,int data,int pos){
struct node *n=malloc(sizeof(struct node));
struct node *temp;
int i;
n->data=data;
if(pos<1){
free(n);
```

```
return head;
}
if(head==NULL){
if(pos!=1){
free(n);
return head;
}
n->next=n;
return n;
}
if(pos==1){
temp=head;
while(temp->next!=head)temp=temp->next;
n->next=head;
temp->next=n;
return n;
}
temp=head;
for(i=1;i<pos-1&&temp->next!=head;i++)temp=temp->next;
if(i!=pos-1){
free(n);
return head;
}
n->next=temp->next;
temp->next=n;
return head;
```

```
}
```

4.Find the 2nd largest element in a queue.

```
#include<stdio.h>
int main(){
int q[]={10,30,20,50,40};
int n=5,largest,second,i,found=0,found2=0;
for(i=0;i<n;i++){
if(!found||q[i]>largest){
if(found){
second=largest;
found2=1;
}
largest=q[i];
found=1;
}else if(q[i]<largest&&(!found2||q[i]>second)){
second=q[i];
found2=1;
}
}
if(found2)printf("%d",second);
else printf("No second largest element");
return 0;
}
```

5.Insert 3rd element and check whether 1st and 3rd element are same in a Linked List.

```
#include<stdio.h>
struct node{
```

```
int data;
struct node *next;
};
int check(struct node *head){
if(head==NULL||head->next==NULL||head->next->next==NULL)return 0;
return head->data==head->next->next->data;
}
```

6.Implement two stacks in one array write routine to traverse the array and display.

```
#include<stdio.h>
#define MAX 10
int a[MAX];
int top1=-1,top2=MAX;
void push1(int x){
if(top1+1==top2)printf("Overflow\n");
else a[++top1]=x;
}
void push2(int x){
if(top1+1==top2)printf("Overflow\n");
else a[--top2]=x;
}
void pop1(){
if(top1==-1)printf("Underflow\n");
else printf("%d\n",a[top1--]);
}
void pop2(){
if(top2==MAX)printf("Underflow\n");
```

```
else printf("%d\n",a[top2++]);
}
void display(){
int i;
printf("Stack 1: ");
for(i=top1;i>=0;i--)printf("%d ",a[i]);
printf("\nStack 2: ");
for(i=top2;i<MAX;i++)printf("%d ",a[i]);
printf("\n");
}
int main(){
push1(10);
push1(20);
push2(30);
push2(40);
display();
pop1();
pop2();
display();
return 0;
}
```

7.Given a Linked List containing n head nodes where every node in the Linked List contains two pointers next pointer to next node in the Linked List bottom pointer to the sub linked list where the current node is the head and each of the sub linked list nodes and these nodes sorted in ascending order based on their data. We need to appear in a singly Linked list while maintaining sorted order.

```
#include<stdio.h>
```

```
#include<stdlib.h>
struct node{
int data;
struct node *next,*down;
};
struct node *merge(struct node *a,struct node *b){
struct node dummy;
struct node *tail=&dummy;
dummy.down=NULL;
while(a!=NULL&&b!=NULL){
if(a->data<=b->data){
tail->down=a;
a=a->down;
}else{
tail->down=b;
b=b->down;
}
tail=tail->down;
}
tail->down=(a!=NULL)?a:b;
return dummy.down;
}
struct node *flatten(struct node *head){
if(head==NULL||head->next==NULL)return head;
head->next=flatten(head->next);
head=merge(head,head->next);
```

```
return head;
}
void display(struct node *head){
while(head!=NULL){
printf("%d ",head->data);
head=head->down;
}
}
```

8.Given k sorted Linked List of different sizes , we need to merge them into a single list maintaining their sorted order.

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
struct node *next;
};
struct node *merge(struct node *a,struct node *b){
struct node dummy;
struct node *tail=&dummy;
dummy.next=NULL;
while(a!=NULL&&b!=NULL){
if(a->data<=b->data){
tail->next=a;
a=a->next;
}else{
tail->next=b;
```

```
b=b->next;
}
tail=tail->next;
}
tail->next=(a!=NULL)?a:b;
return dummy.next;
}
struct node *mergeK(struct node *list[],int k){
int i;
if(k==0)return NULL;
while(k>1){
int j=0;
for(i=0;i+1<k;i+=2)list[j++]=merge(list[i],list[i+1]);
if(k%2==1)list[j++]=list[k-1];
k=j;
}
return list[0];
}
void display(struct node *head){
while(head!=NULL){
printf("%d ",head->data);
head=head->next;
}
}
```

9.Swap kth node from beginning with kth node from end in a Linked List.

```
#include<stdio.h>
```

```
#include<stdlib.h>
struct node{
int data;
struct node *next;
};
struct node *swapK(struct node *head,int k){
struct node *x,*y,*px=NULL,*py=NULL,*temp,*cur=head;
int n=0,i;
while(cur!=NULL){
n++;
cur=cur->next;
}
if(k<1||k>n)return head;
if(2*k-1==n)return head;
x=head;
for(i=1;i<k;i++){
px=x;
x=x->next;
}
y=head;
for(i=1;i<n-k+1;i++){
py=y;
y=y->next;
}
if(px!=NULL)px->next=y;
else head=y;
```

```
if(py!=NULL)py->next=x;
else head=x;
temp=x->next;
x->next=y->next;
y->next=temp;
return head;
}
void display(struct node *head){
while(head!=NULL){
printf("%d ",head->data);
head=head->next;
}
}
```

10.Multiply two numbers representer by two Linked List .

```
#include<stdio.h>
#include<stdlib.h>
struct node{
int data;
struct node *next;
};
long long getNumber(struct node *head){
long long num=0;
while(head!=NULL){
num=num*10+head->data;
head=head->next;
}
```

```
return num;
}
struct node *createDigit(int digit){
struct node *n=malloc(sizeof(struct node));
n->data=digit;
n->next=NULL;
return n;
}
struct node *multiply(struct node *a,struct node *b){
long long product=getNumber(a)*getNumber(b);
struct node *result=NULL;
if(product==0)return createDigit(0);
while(product>0){
struct node *n=createDigit(product%10);
n->next=result;
result=n;
product/=10;
}
return result;
}
void display(struct node *head){
while(head!=NULL){
printf("%d",head->data);
head=head->next;
}
printf("\n");
```

}